

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсового проекта
ассистент кафедры информатики
_____ А.Г. Бокун
____.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:
**«ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ГЕНЕРАЦИИ ИЗОБРАЖЕНИЙ
ФРАКТАЛЬНОГО ПЛАМЕНИ»**

БГУИР КП 6-05 0612 02 26 ПЗ

Выполнил студент группы 453501
СКРОБОТ Денис Алексеевич

(подпись студента)

Курсовой проект представлен на
проверку _____._____.2026

(подпись студента)

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Анализ аналогов	6
1.2 Формирование системных требований	9
2 Анализ литературы	10
2.1 Принципы объектно-ориентированного проектирования	10
2.2 Принципы SOLID и качество кода	10
2.3 Шаблоны проектирования	10
2.4 Моделирование предметной области	11
2.5 Слоистая организация приложения	11
2.6 Контракты и переносимость моделей	12
3 Проектирование системы	13
3.1 Общая архитектура	13
3.2 Применяемые технологии	14
3.3 Frontend	19
3.4 API gateway	22
3.5 IAM	23
3.6 Generator	24
3.7 Развёртывание	28
4 Тестирование программного обеспечения	30
4.1 Стратегия тестирования	30
4.2 Сценарии тестирования	30
Заключение	31
Список использованных источников	32
Приложение А (обязательное) Листинг программного кода	34

ВВЕДЕНИЕ

Фрактальная графика относится к области визуализации сложных математических моделей и активно используется в цифровом искусстве, дизайне и научной визуализации. Одним из наиболее выразительных направлений является генерация фрактального «пламени», предложенная С. Дрейвсом, где изображение формируется итеративными нелинейными преобразованиями и последующей тональной обработкой плотности. Благодаря богатству форм и высокой детализации такие изображения востребованы как в творческих проектах, так и в обучении математическому моделированию.

Современные инструменты генерации фракталов чаще представлены настольными приложениями и ориентированы на локальную работу пользователя. При этом растёт спрос на веб-сервисы, обеспечивающие доступ с разных устройств, хранение истории и возможность публикации результатов. Для решения этих задач требуется система, сочетающая удобный пользовательский интерфейс, вычислительно ёмкий рендеринг и надёжное хранение артефактов.

Целью курсового проекта является создание распределённой системы генерации фракталов с веб-интерфейсом, поддержкой аутентификации пользователей и хранением истории созданных работ. Архитектура системы построена на принципах микросервисов, что облегчает масштабирование и независимое развитие компонентов. Разработка выполнена в соответствии с принципами объектно-ориентированного проектирования и ориентирована на практическое применение современных инструментов сборки и развёртывания.

Для достижения поставленной цели были решены следующие задачи:

- анализ предметной области;
- проектирование микросервисной архитектуры, формирование функциональных и нефункциональных требований к системе;
- выбор технологий реализации, сборки, развёртывания и программная реализация системы;
- тестирование ключевых сценариев работы пользователей.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ аналогов

Для определения требований и позиционирования проекта были рассмотрены наиболее известные решения в области генерации фрактального пламени. Сравнение выполнено по критериям: доступность (desktop/web), качество рендеринга, удобство настройки параметров, наличие истории и механизмов публикации результатов.

1.1.1 Apophysis — один из наиболее известных редакторов фрактального пламени. Программа распространяется как open-source и предоставляет богатый набор инструментов для ручной настройки трансформаций, палитр и параметров рендера [1]. К достоинствам можно отнести простоту использования и обилие обучающих материалов. Основные ограничения связаны с настольным форматом, отсутствием встроенного хранения истории и невозможностью работы в распределённой среде. Интерфейс Apophysis показан на рисунке 1.1.

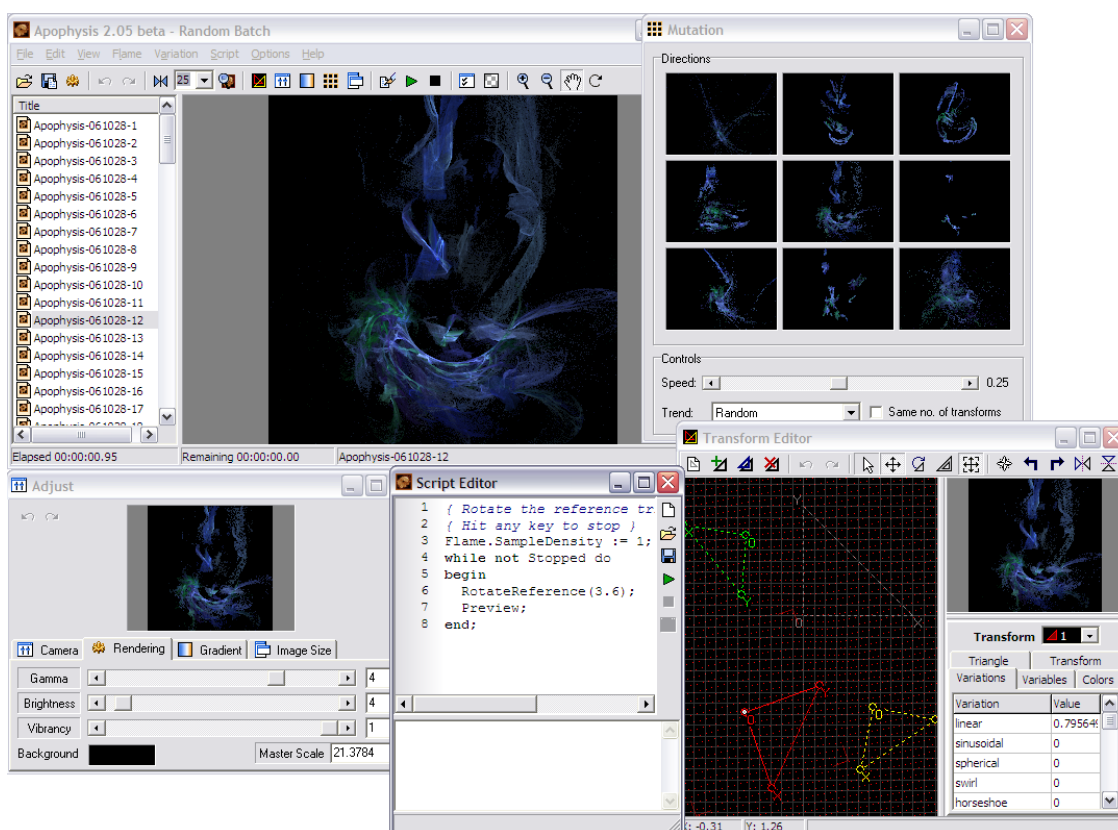


Рисунок 1.1 – Скриншот пользовательского интерфейса Apophysis с несколькими открытыми окнами

Дополнительно стоит отметить, что основная работа в Apophysis строится вокруг ручного подбора вариаций и параметров, поэтому пользователь часто вынужден сохранять множество промежуточных файлов.

Отсутствие центрального хранилища и единой учётной записи делает обмен результатами и совместную работу затруднительными.

1.1.2 JWildfire — мощный кроссплатформенный редактор фрактального пламени с большим набором вариаций и встроенными генераторами случайных фракталов [2]. Решение предоставляет широкие возможности, но требует значительного времени на освоение и ориентировано на опытных пользователей. Встроенные механизмы совместной работы и веб-публикации отсутствуют, а хранение результатов полностью локально. Интерфейс JWildfire представлен на рисунке 1.2.

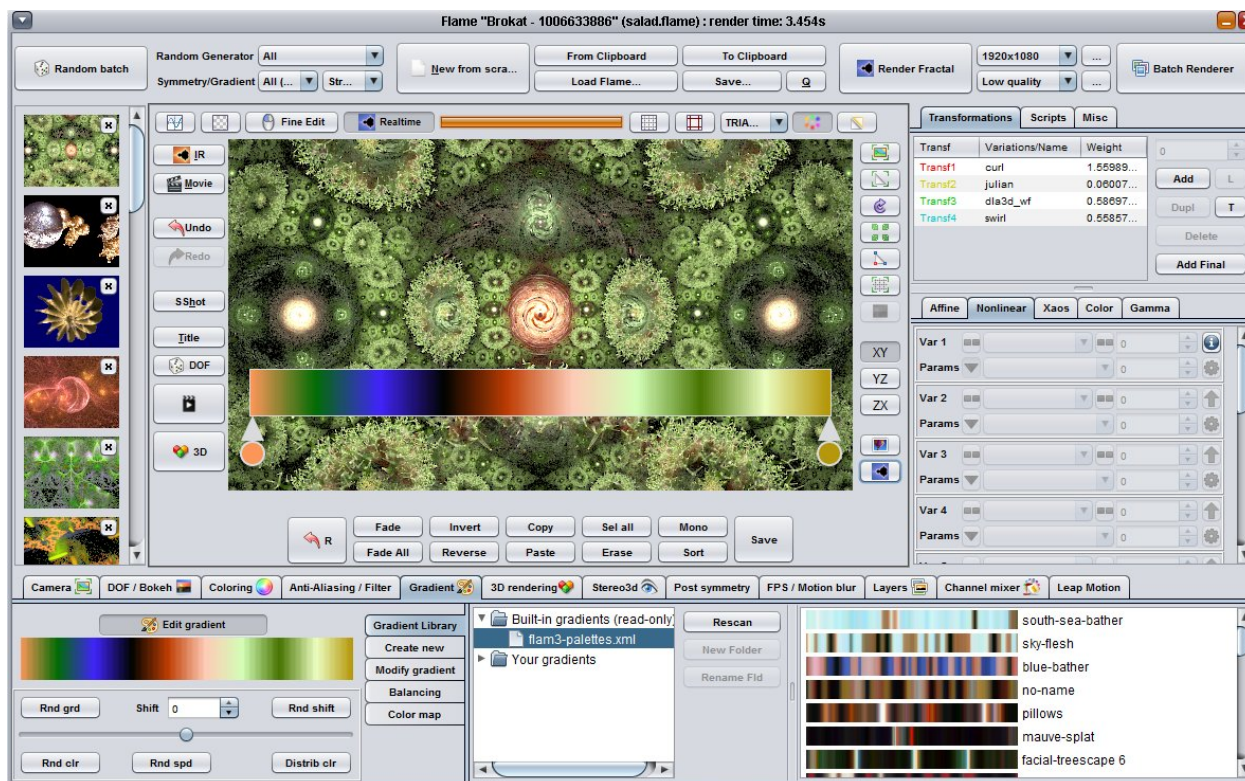


Рисунок 1.2 – Скриншот пользовательского интерфейса JWildfire

Сильной стороной JWildfire является гибкость и глубина настроек, что делает его удобным для экспериментов и профессионального использования. Однако сложный интерфейс и необходимость локальной установки снижают доступность для начинающих пользователей и не решают задачу быстрого доступа к проектам с разных устройств.

1.1.3 Chaotica — коммерческое приложение, ориентированное на получение высококачественных изображений и ускоренный рендеринг (включая GPU) [3]. Преимущества: высокая скорость и качество, развитый инструментарий анимации. Недостатки: закрытость, платная модель распространения и ориентация на локальную работу без интегрированного облачного хранения и пользовательских профилей. Пример рабочего окна Chaotica приведён на рисунке 1.3.

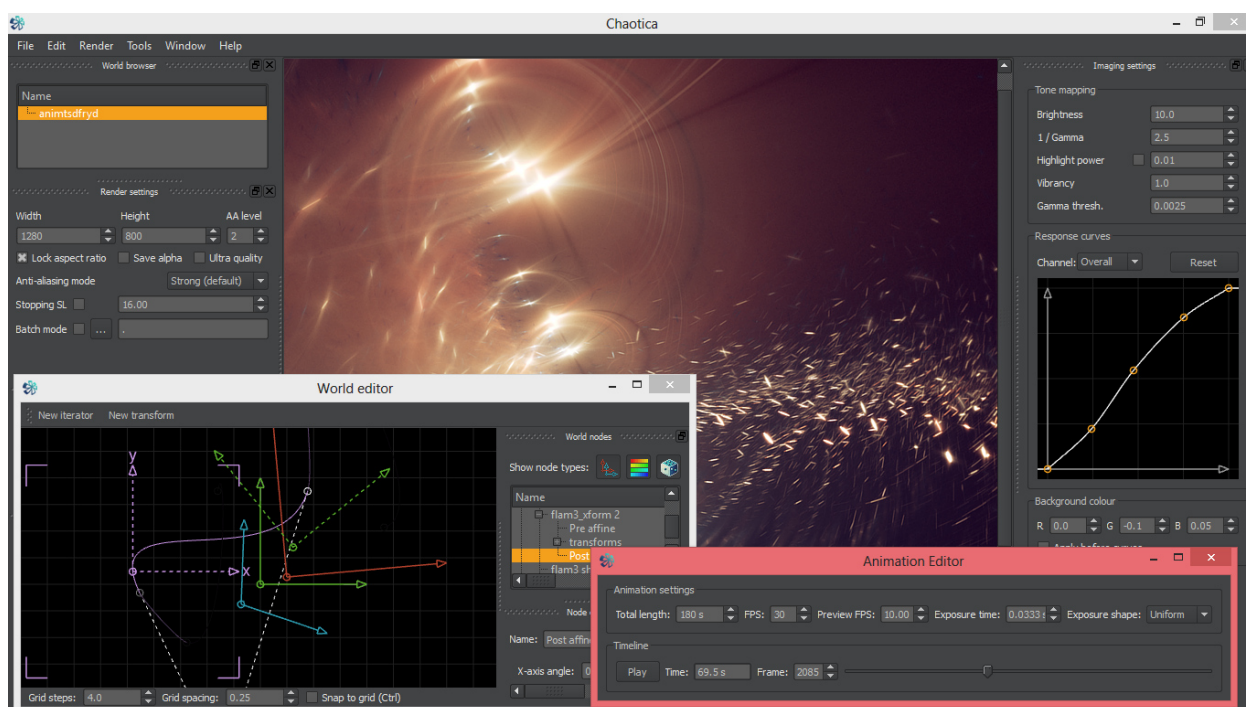


Рисунок 1.3 – Скриншот анимации фрактала, создаваемой в Chaotica

В практической эксплуатации продукт удобен для студийных задач и финального качества рендера, однако ориентирован на автономное использование. Это ограничивает возможность модификаций, интеграции с веб-архивом и построения персонализированного профиля пользователя.

1.1.4 Сводное сравнение аналогов приведено в таблице 1.1.

Таблица 1.1 – Сравнение аналогов по ключевым критериям

Продукт	Сильные стороны	Ограничения
Ароphysis	Простота использования, open-source, богатая база примеров	Только desktop, нет истории и веб-доступа
JWildfire	Большое количество вариаций, кросс-платформенность, мощный набор функций	Высокая сложность интерфейса, локальное хранение, отсутствие веб-режима
Chaotica	Высокое качество рендера, поддержка GPU, развитая анимация	Платная лицензия, закрытость, отсутствие облачной истории

Анализ показал, что существующие решения преимущественно ориентированы на локальный рабочий процесс и не обеспечивают удобной работы с историей и публикацией результатов. Это определило требования к проектируемой системе, в частности необходимость веб-интерфейса, учётных записей и долговременного хранения рендеров.

Следовательно, ключевым конкурентным преимуществом будущей системы становится доступность из браузера и единое пространство проектов, где результаты сохраняются и доступны для повторного использования.

1.2 Формирование системных требований

На основе анализа аналогов сформированы функциональные требования к системе генерации фракталов:

- регистрация, вход и обновление пользовательской сессии с использованием JWT-токенов;
- создание задачи генерации фрактала с передачей размеров изображения, количества итераций, уровня симметрии, гамма-коррекции, набора функций и аффинных преобразований;
- получение списка доступных функций фрактального пламени;
- отображение прогресса генерации и промежуточного предпросмотра;
- получение списка ранее созданных фракталов с пагинацией, сортировкой и фильтрацией по пользователю;
- просмотр отдельного фрактала, его параметров и итогового изображения;
- отображение главной страницы с формой генерации и feed последних сгенерированных фракталов с подписями пользователей;
- отображение страницы профиля с редактированием пользовательской информации и историей созданных фракталов;
- сохранение параметров генерации в PostgreSQL и сохранение PNG-изображений в S3-совместимом хранилище MinIO;
- редактирование базовых данных профиля пользователя.

Сформированы также нефункциональные требования:

- производительность: приемлемое время рендера при стандартных параметрах качества;
- масштабируемость: возможность независимо увеличивать количество экземпляров gateway и generator без изменения клиентского приложения;
- надёжность: сохранность истории генераций в PostgreSQL и изображений в MinIO при перезапуске контейнеров;
- безопасность: stateless-аутентификация через JWT, передача токена в HTTP-заголовке и проксирование его во внутренние gRPC-вызовы;
- переносимость: развёртывание всех компонентов через Docker Compose;
- сопровождаемость: разделение frontend, API gateway, IAM и generator по самостоятельным зонам ответственности.

2 АНАЛИЗ ЛИТЕРАТУРЫ

2.1 Принципы объектно-ориентированного проектирования

Объектно-ориентированный подход опирается на понятия инкапсуляции, наследования, полиморфизма и абстракции. Эти принципы позволяют моделировать предметную область в виде взаимосвязанных классов, обеспечивая повторное использование и расширяемость кода [4]. Для разрабатываемой системы это особенно важно, поскольку логика генерации фракталов включает множество разновидностей преобразований, параметров рендера и вариаций, которые удобно представлять в виде иерархий объектов.

В проекте объектные модели описывают параметры рендера, состояние проекта и пользовательский контекст, что позволяет разделить вычислительную и интерфейсную части. Это упрощает развитие функциональности и делает код более устойчивым к изменениям.

2.2 Принципы SOLID и качество кода

Принципы SOLID ориентированы на снижение связности и повышение устойчивости к изменениям. В частности, разделение ответственности и открытость к расширению позволяют добавлять новые алгоритмы генерации и модули без переработки существующего кода [5]. Эти принципы были использованы при проектировании сервисов, чтобы обеспечивать независимость компонентов и удобство тестирования.

На практике это выразилось в выделении отдельных интерфейсов для рендера, хранения и пользовательских операций, а также в уменьшении числа прямых зависимостей между слоями. Такой подход упростил модульное тестирование и локализацию ошибок.

2.3 Шаблоны проектирования

В литературе по объектно-ориентированному проектированию выделяются шаблоны, упрощающие решение типовых инженерных задач [6]. Для проекта генерации фрактального пламени особенно важны шаблоны, которые позволяют отделить изменяемые алгоритмы от стабильной инфраструктуры приложения.

Шаблон Strategy применим к вариационным функциям фрактального пламени. Каждая функция преобразует точку по собственному правилу, но внешний код генерации может работать с общим интерфейсом функции. Благодаря этому добавление новой вариации не требует переписывать основной цикл рендера. Достаточно реализовать новую модель функции и зарегистрировать её в фабрике или строителе функций.

Шаблон Factory Method полезен при создании доменных объектов на основе входных DTO или protobuf-сообщений. Внешний контракт передаёт имя функции и числовые параметры, а внутри generator должен появиться объект конкретного класса. Вынесение этой логики из сервиса генерации снижает связность и делает ошибочные имена функций отдельным контролируемым сценарием.

Идея Builder применима к созданию задачи генерации. Задача включает размеры изображения, число итераций, симметрию, гамму, набор функций, набор аффинных преобразований и пользовательский контекст. Последовательная сборка такой конфигурации удобнее, чем передача большого количества несвязанных параметров между методами.

2.4 Моделирование предметной области

Для описания предметной области генерации фракталов применяются подходы доменно-ориентированного проектирования. Они позволяют выделить ключевые сущности и описать их жизненный цикл [7]. В данном проекте основными сущностями являются пользователь, фрактал, вариационная функция, аффинное преобразование, задача генерации и изображение результата.

Сущность фрактала хранит параметры, достаточные для повторного отображения результата: размеры, количество итераций, уровень симметрии, гамму, список функций и список аффинных преобразований. Изображение при этом не смешивается с параметрами: в реляционной базе хранится только имя объекта, а бинарное содержимое помещается в объектное хранилище. Такое разделение поддерживает принцип единственной ответственности на уровне данных.

Пользовательская сущность относится к другому bounded context — IAM. Она не должна зависеть от деталей генерации фракталов. Поэтому generator хранит только внешний идентификатор пользователя, полученный из JWT, а полная информация о профиле остаётся в IAM. Это уменьшает связность микросервисов и упрощает изменение профиля без миграций в сервисе генерации.

2.5 Слоистая организация приложения

Серверные сервисы проекта разделяются на несколько слоёв: транспортный, сервисный, доменный и инфраструктурный. Транспортный слой принимает HTTP или gRPC-запросы и отвечает за преобразование данных из внешнего формата. Сервисный слой координирует сценарии использования: запуск генерации, получение истории, чтение изображения, обновление пользователя. Доменный слой содержит модели фрактала, функций и параметров. Инфраструктурный слой работает с PostgreSQL,

MinIO и внешними библиотеками.

Такое разделение важно для тестируемости. Например, алгоритм генерации можно проверять отдельно от gRPC-сервера, а мапперы DTO можно тестировать без запуска базы данных. Кроме того, слой gateway не обязан знать внутреннее устройство generator: он опирается на контракт и выполняет роль адаптера между HTTP-клиентом и внутренними сервисами.

2.6 Контракты и переносимость моделей

В распределённой системе объектная модель существует сразу в нескольких формах: как доменная модель внутри сервиса, как схема базы данных, как protobuf-сообщение для gRPC и как JSON-модель во внешнем HTTP API. Если эти формы изменяются вручную и независимо, быстро появляются расхождения между клиентом и сервером.

Для снижения этого риска используются формальные контракты. gRPC-контракт generator описывает внутренние сообщения и сервисные методы, а OpenAPI-спецификация описывает внешний HTTP API gateway. Кодогенерация по этим контрактам позволяет получать типы, интерфейсы и модели из единого источника. Это не отменяет ручное проектирование доменной модели, но делает границы между сервисами более проверяемыми.

3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1 Общая архитектура

Общая архитектура системы показана на рисунке 3.1.

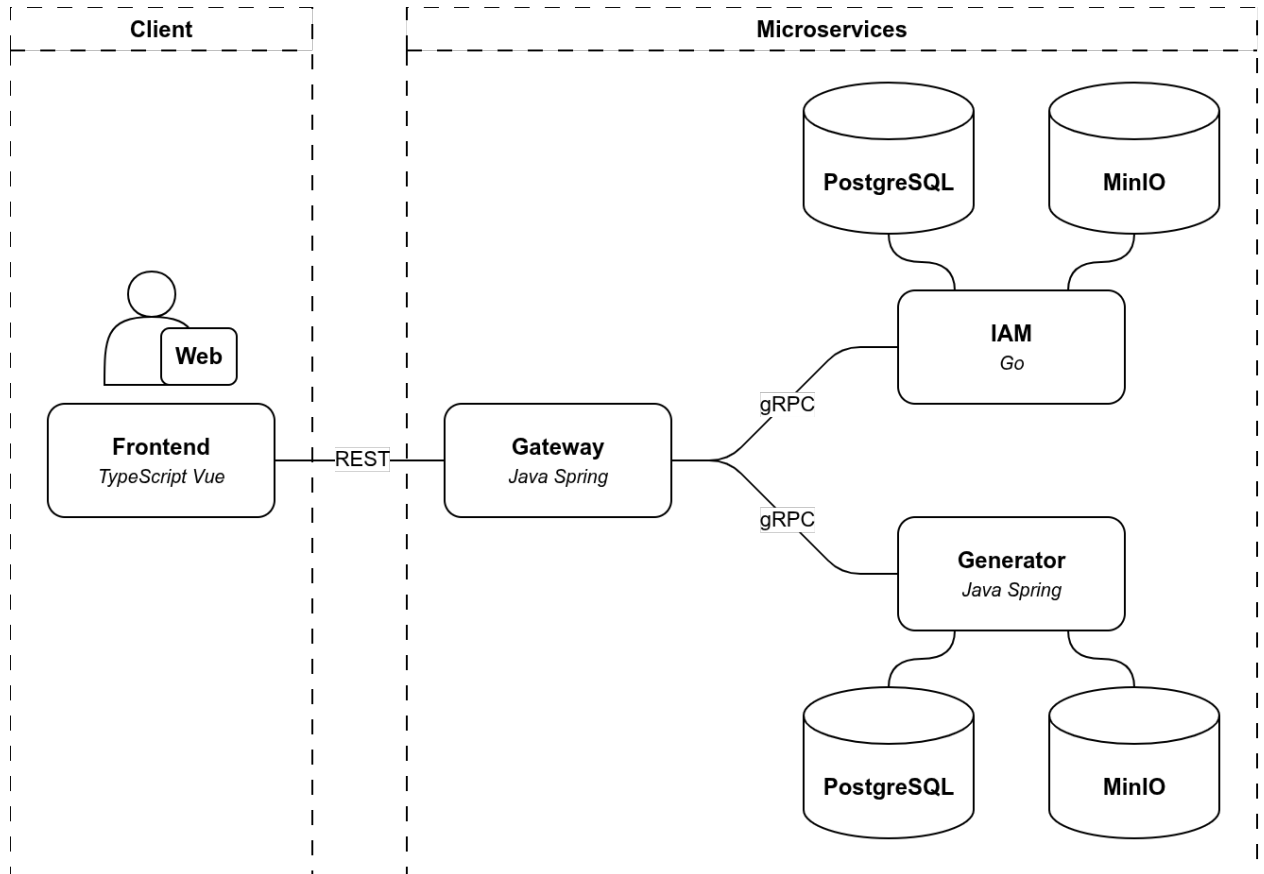


Рисунок 3.1 – Диаграмма архитектуры

Проект реализуется как распределённая система из четырёх компонентов:

- frontend — клиентское приложение на TypeScript, Vue, Tailwind CSS и Axios;
- API gateway — единая HTTP-точка входа для клиентского приложения, реализованная на Java и Spring;
- IAM — сервис управления пользователями и аутентификацией, реализованный на Go;
- generator — сервис генерации и хранения фракталов, реализованный на Java и Spring.

Frontend взаимодействует только с API gateway по HTTP. Gateway скрывает внутреннюю структуру backend-части и обращается к IAM и generator по gRPC [8]. Такое разделение позволяет независимо развивать пользовательский интерфейс, аутентификацию и вычислительную часть.

Внешний контракт остаётся ориентированным на браузерное приложение, а внутренние сервисы могут использовать более строгие protobuf-контракты.

Зоны ответственности компонентов представлены в таблице 3.1.

Таблица 3.1 – Ответственность компонентов системы

Компонент	Назначение	Взаимодействие
Frontend	Отображение интерфейса, ввод параметров, просмотр галереи и профиля	HTTP-запросы к gateway, получение событий прогресса и изображений
API gateway	Единая точка входа и адаптер между HTTP API и внутренними сервисами	HTTP с frontend, gRPC-вызовы IAM и generator, передача JWT во внутренние metadata
IAM	Регистрация, вход, обновление токенов и хранение профиля пользователя	gRPC API для gateway, хранение пользовательской записи в PostgreSQL
Generator	Генерация фрактального пламени, история работ, хранение изображений	gRPC API для gateway, PostgreSQL для параметров, MinIO для PNG-файлов

3.2 Применяемые технологии

3.2.1 Язык программирования Java является объектно-ориентированным языком, разработанным компанией Sun Microsystems и в настоящее время поддерживаемым корпорацией Oracle. Java обеспечивает высокий уровень абстракции, платформенную независимость и безопасность, что делает его одним из самых популярных языков в мире.

Одним из основных преимуществ Java является его независимость от платформы, достигаемая благодаря использованию виртуальной машины Java (JVM), которая исполняет байт-код, одинаковый для всех операционных систем. Это свойство позволяет создавать переносимые приложения без необходимости модификации исходного кода.

Java поддерживает основные принципы объектно-ориентированного программирования, такие как наследование, инкапсуляция, полиморфизм и абстракция. Эти особенности обеспечивают высокую модульность и повторное использование кода [9].

В рамках проекта Java используется в двух сервисах: API gateway и generator. Для этих компонентов язык выбран благодаря следующим преимуществам:

- наличие развитой экосистемы для HTTP API, gRPC, работы с базами

данных и контейнеризированного запуска;

- развитая экосистема, включающая библиотеки и фреймворки, ускоряющие процесс разработки;

- встроенная система управления памятью с автоматическим сборщиком мусора, упрощающая разработку и снижающая вероятность ошибок, связанных с управлением ресурсами;

- высокая читаемость и строгая типизация, что способствует написанию надёжного и легко сопровождаемого кода.

В gateway Java применяется для обработки HTTP-запросов, настройки фильтра JWT и вызова gRPC-сервисов. В generator язык используется для реализации алгоритма фрактального пламени, очереди задач, сохранения метаданных и работы с S3-совместимым хранилищем.

3.2.2 Spring — это мощный и гибкий фреймворк с открытым исходным кодом для разработки корпоративных и веб-приложений на языке Java. Благодаря своей модульной структуре, высокой производительности и удобству масштабирования, Spring широко применяется при создании распределённых и высоконагруженных систем, включая веб-сервисы и многопользовательские онлайн-платформы.

Одной из ключевых особенностей Spring является внедрение зависимостей (Dependency Injection), которое способствует слабой связанности компонентов приложения и облегчает их тестирование, повторное использование и расширение. Фреймворк построен по принципу инверсии управления (Inversion of Control), где управление жизненным циклом объектов осуществляется специальным контейнером.

В проекте Spring используется в gateway и generator. Gateway принимает HTTP-запросы от frontend, а generator предоставляет gRPC API для генерации фракталов и доступа к истории.

В процессе разработки были использованы следующие модули Spring:

- Spring Boot — модуль, предназначенный для упрощения конфигурации и быстрой инициализации проекта. Он предоставляет встроенный сервер и автоматическую настройку компонентов, что позволило сосредоточиться на реализации логики приложения;

- Spring Web — модуль, реализующий обработку HTTP-запросов, маршрутизацию и построение REST API gateway;

- Spring gRPC — модуль для реализации gRPC-сервера в generator и gRPC-клиента в gateway;

- Spring Data JPA — модуль для работы с PostgreSQL через объектно-реляционное отображение;

- Spring Security — модуль, обеспечивающий обработку JWT в HTTP-запросах gateway и gRPC metadata generator.

Использование Spring позволило добиться высокой скорости обработки запросов, надёжности и масштабируемости серверной части. Благодаря модульной архитектуре фреймворка, разработка велась по

принципу разделения ответственности, что улучшило структуру проекта и упростило поддержку и развитие функционала [10].

Дополнительно применяются встроенные механизмы валидации данных, централизованная обработка ошибок и healthcheck'и, что обеспечивает единый формат ответов и предсказуемость поведения сервисов.

3.2.3 Gradle используется для сборки Java-сервисов gateway и generator. Он обеспечивает управление зависимостями, генерацию Java-кода из protobuf-контрактов, запуск тестов и сборку исполняемых приложений. В gateway дополнительно подключён OpenAPI Generator: из файла `openapi.yaml` формируются Java-модели и API-интерфейсы, которые затем реализуются контроллерами сервиса [11].

3.2.4 Язык Go используется для реализации IAM-сервиса. Для сервиса аутентификации важны быстрый запуск, простая модель конкурентного выполнения, компактная поставка и удобная работа с сетевыми сервисами. Go хорошо подходит для небольших backend-компонентов, которые предоставляют API, работают с базой данных и не требуют тяжёлой runtime-среды [12].

В рамках проекта IAM отвечает за регистрацию пользователей, проверку пароля, выпуск JWT и обновление сессии. Выделение этого сервиса на Go позволяет развивать пользовательскую часть независимо от Java-сервисов генерации.

3.2.5 Frontend реализуется как одностраничное веб-приложение. TypeScript добавляет статическую типизацию поверх JavaScript и помогает согласовать модели запросов и ответов с OpenAPI-спецификацией [13]. На основе OpenAPI-контракта формируются типы и клиентские функции для обращения к gateway, поэтому изменения HTTP API быстрее обнаруживаются на этапе сборки frontend. Vue выбран как фреймворк для построения реактивного интерфейса: он упрощает разбиение экрана на компоненты, управление состоянием формы генерации и обновление истории фракталов [14].

Tailwind CSS применяется для быстрой сборки адаптивного интерфейса из утилитарных классов [15]. Такой подход удобен для страниц с большим количеством параметров генерации: размеры, отступы, состояния кнопок и сетки можно задавать непосредственно рядом с разметкой компонента.

Axios используется как HTTP-клиент для обращений frontend к API gateway [16]. Через него выполняются запросы регистрации и входа, получение списка фракталов, запуск генерации и загрузка изображений. Для прогресса рендера применяется поток событий `text/event-stream`.

3.2.6 Для хранения структурированных данных используется система управления базами данных PostgreSQL [17]. Она выбрана из-за поддержки транзакций, ограничений целостности, индексов и зрелой интеграции с

backend-стеком. Конкретная схема данных generator и IAM описана выше в подразделах о моделях данных, поэтому в данном месте фиксируется только обоснование выбора СУБД.

3.2.7 Для хранения изображений используется MinIO — S3-совместимое объектное хранилище [18]. Оно позволяет отделить бинарные PNG-файлы от реляционных данных и оставить PostgreSQL для метаданных, связей и поиска. Благодаря совместимости с S3 при необходимости локальное хранилище можно заменить внешним объектным хранилищем без изменения пользовательских сценариев.

3.2.8 gRPC применяется для внутреннего взаимодействия gateway с generator и IAM [8]. По сравнению с передачей JSON между backend-сервисами protobuf-контракты дают строгую схему сообщений, компактное представление данных и удобную генерацию клиентского и серверного кода. Подробный сценарий потоковой передачи прогресса рассмотрен в подразделе о внутренних gRPC-контрактах.

3.2.9 JSON Web Token используется как формат access token [19]. Токен содержит идентификатор пользователя и имя, подписывается секретным ключом и проверяется без обращения к базе данных при каждом запросе. Благодаря этому gateway остаётся stateless, а механизм передачи пользовательского контекста между сервисами не требует общей HTTP-сессии.

3.2.10 Для развёртывания использовалась контейнеризация Docker. Каждый микросервис упакован в отдельный образ, а общий сценарий запуска описан в Docker Compose [20]. Compose выбран как достаточный для курсового проекта способ поднять приложение, базу данных и MinIO одной командой в воспроизводимом локальном или серверном окружении.

3.2.11 Visual Studio Code (VS Code) — это лёгкая, но мощная интегрированная среда разработки, разработанная компанией Microsoft. С момента своего выхода в 2015 году она завоевала огромную популярность благодаря простоте, скорости и гибкости. VS Code является кроссплатформенным инструментом, доступным на Windows, macOS и Linux, что делает его универсальным решением для разработчиков по всему миру [21].

Экран редактирования кода в Visual Studio Code показан на рисунке 3.2.

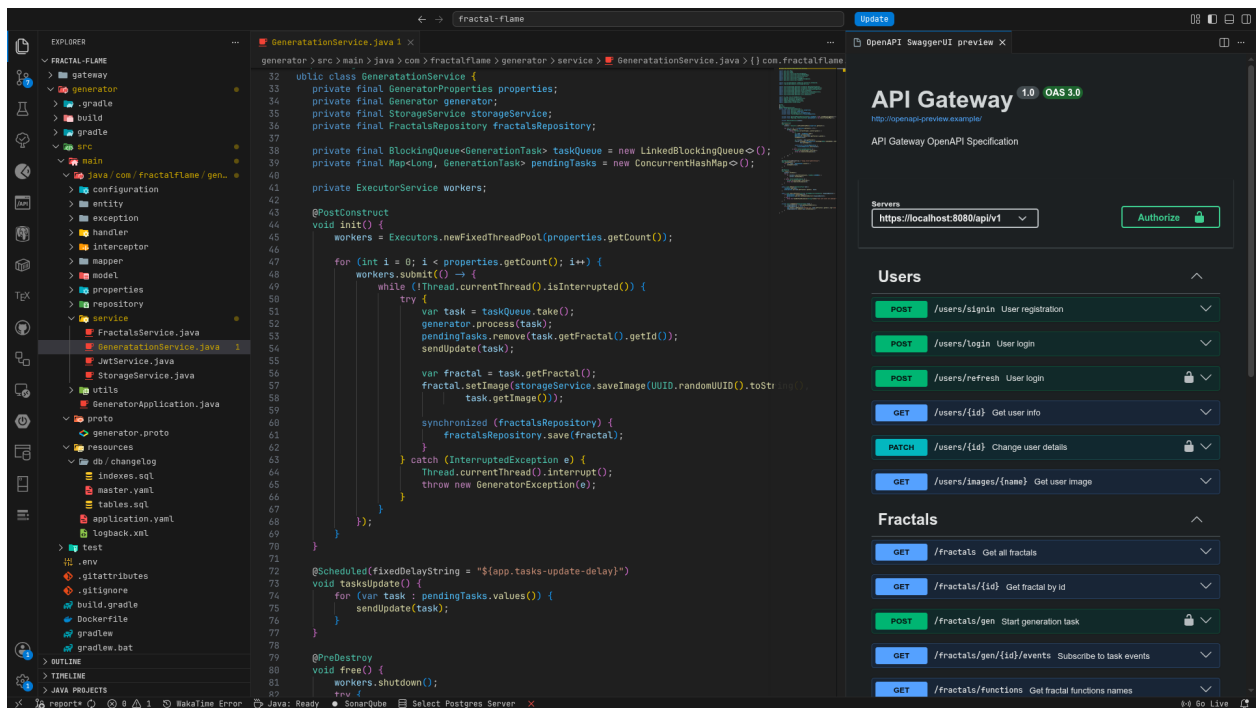


Рисунок 3.2 – Экран редактирования кода Visual Studio Code

VS Code не перегружена лишними функциями, но в то же время предоставляет всё необходимое для разработки: интеллектуальную подсветку синтаксиса, автодополнение, отладку, поддержку Git, систему контроля версий и встроенный терминал. Это позволяет разрабатывать и тестировать код быстро и эффективно, без лишних задержек.

Основные причины выбора VS Code включают:

- 1 Лёгкость и быстродействие. VS Code быстро запускается и работает с минимальными задержками, что делает её идеальной для разработки фронтенд-части и быстрого прототипирования.
- 2 Расширяемость. Благодаря множеству доступных расширений, VS Code легко адаптируется под любые задачи.
- 3 Интеграция с Git и другими системами контроля версий. VS Code имеет встроенную поддержку Git, что упрощает работу с репозиториями и позволяет эффективно управлять версиями проекта.

Разработка всех компонентов проекта выполнялась исключительно в Visual Studio Code, что обеспечило единообразие среды и повторяемость процесса. Дополнительно использовались расширения для Java, Docker и работы с $\text{\LaTeX}$, что позволило вести и разработку, и оформление документации в одном инструменте. Даже этот документ написан в VS Code с использованием $\text{\LaTeX}$.

3.3 Frontend

Frontend отвечает за пользовательский сценарий работы с системой. Он реализуется как одностраничное приложение на Vue и состоит из маршрутов главной страницы, входа, регистрации и личного кабинета. Общий контейнер приложения содержит шапку с навигацией и состоянием авторизации, нижний колонтитул и область маршрутизатора. Для авторизованного пользователя в шапке отображается переход в профиль и кнопка выхода; для неавторизованного — переходы на страницы входа и регистрации. Дизайн был вдохновлен главным меню игры *Ultrakill* и спроектирован с использованием современных практик [22].

Главный экран предназначен для генерации фрактального пламени. Его внешний вид представлен на рисунке 3.3. Центральную часть занимает полноэкранный просмотр результата, который до запуска задачи показывает заглушку, а во время генерации обновляется base64-предпросмотром из SSE-потока. В нижней правой части размещены кнопка запуска генерации и кнопка открытия панели параметров. Если генерация уже идёт, та же кнопка позволяет остановить получение событий от текущей задачи.

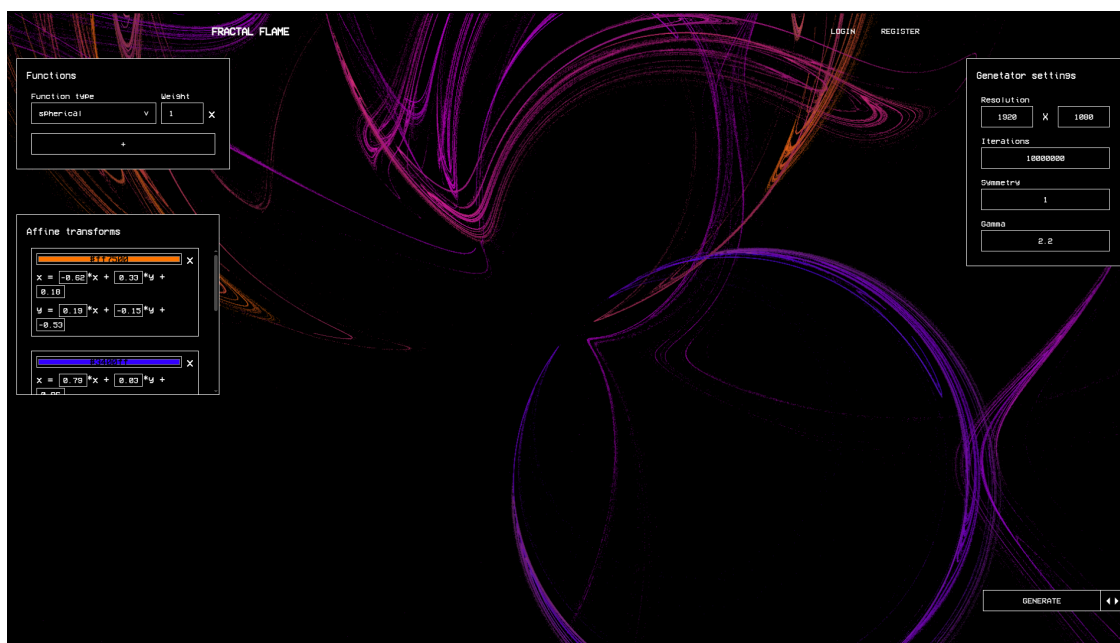


Рисунок 3.3 – Страница генерации фрактального пламени

Параметры генерации разделены между двумя боковыми панелями. Левая панель содержит выбор вариационных функций и наборов аффинных преобразований, включая веса функций, цвет и коэффициенты a , b , c , d , e , f . Список допустимых функций загружается с маршрута `/fractals/functions`, поэтому frontend не хранит отдельный жёстко заданный справочник. Правая панель содержит общие параметры рендера: ширину, высоту, число итераций, уровень симметрии и гамма-коррекцию.

Перед отправкой формы значения проверяются через `vue-validate` и `yup`: пользователь не может отправить пустой список функций или аффинных преобразований, отрицательные веса, слишком маленькое или слишком большое разрешение.

После успешной проверки frontend отправляет POST-запрос на `/fractals/gen`. В ответ приходит идентификатор задачи и начальное состояние прогресса. Затем клиент открывает EventSource на маршруте `/fractals/gen/{id}/events` и подписывается на события `update`, `complete` и `error`. Полученные сообщения обновляют числовой прогресс и изображение предпросмотра, а ошибки показываются пользователю через уведомления.

Под главным экраном находится общая галерея последних работ, показанная на рисунке 3.4. Компонент галереи получает страницу фракталов с сортировкой по дате создания, отображает изображения по маршруту `/fractals/images/{name}` и дополнительно запрашивает сведения о пользователях, чтобы подписать карточки именами авторов. Для навигации по истории используется пагинация.

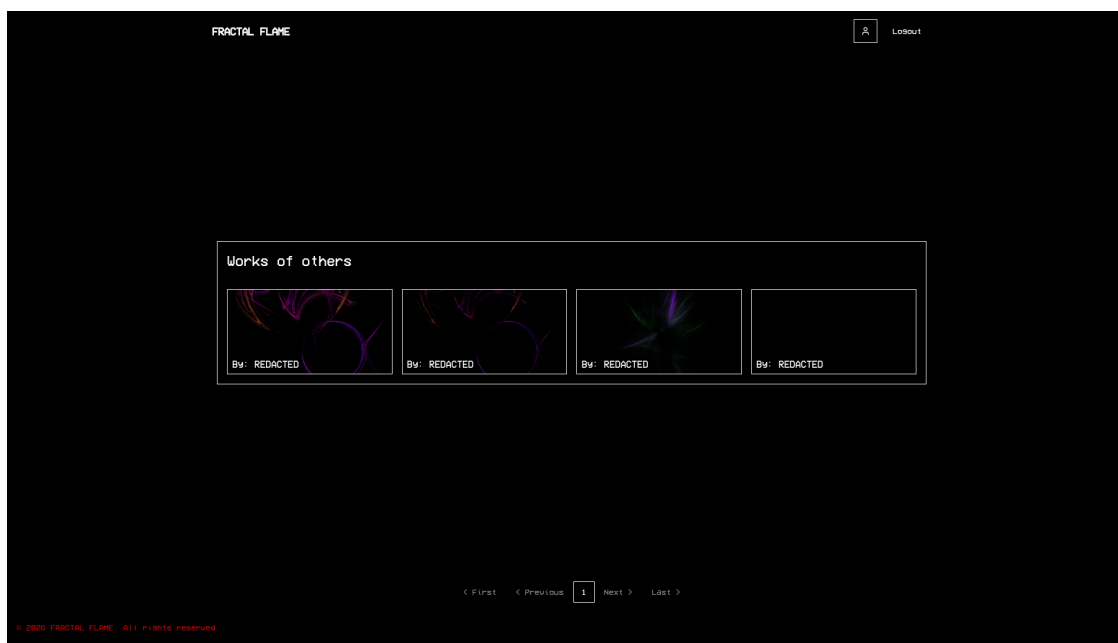


Рисунок 3.4 – Галерея последних сгенерированных фракталов

Сценарии входа и регистрации вынесены на отдельные страницы. Страница входа приведена на рисунке 3.5. Пользователь вводит имя и пароль, после чего frontend отправляет запрос на `/users/login`. Access token сохраняется в локальном хранилище браузера и добавляется Axios-interceptor'ом в заголовок Authorization. Refresh token устанавливается gateway в cookie и используется при автоматическом обновлении access token после ответа 401. Ошибки валидации и неверные учётные данные

отображаются рядом с полями формы.

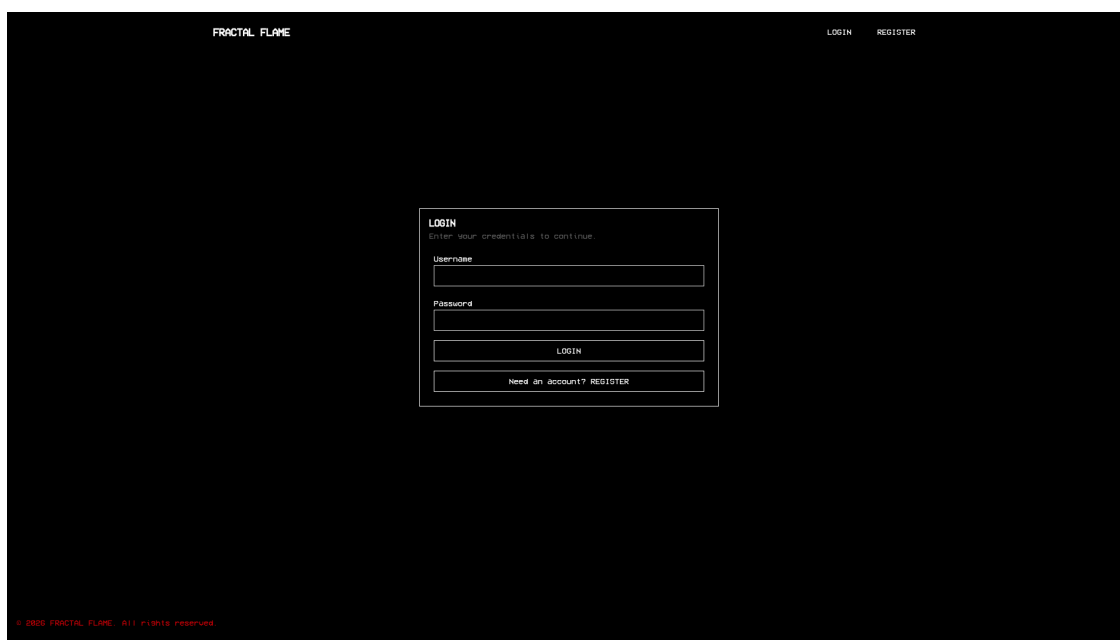


Рисунок 3.5 – Страница входа пользователя

Личный кабинет предназначен для управления профилем и личной историей, его внешний вид показан на рисунке 3.6. В верхней части страницы пользователь может изменить имя, задать новый пароль и загрузить изображение профиля. Перед отправкой изображение читается как base64-строка, проверяется на стороне клиента и передаётся в запросе PATCH /users. Ниже отображается галерея «Your works», где используется тот же компонент карточек, что и в общей галерее, но запрос фильтруется по идентификатору текущего пользователя.

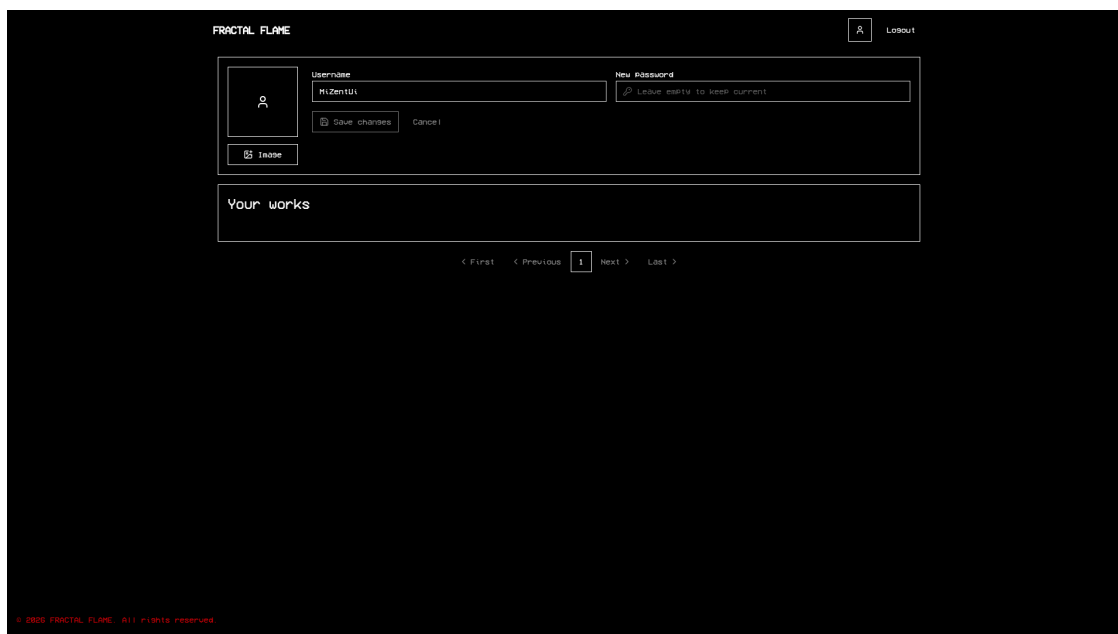


Рисунок 3.6 – Страница профиля пользователя

Клиентский код не обращается напрямую к IAM или generator. Все запросы идут через API gateway, что упрощает хранение базового URL, обработку ошибок и передачу JWT. HTTP-контракт описывается OpenAPI-спецификацией [23]; на стороне frontend она используется как источник типизированных моделей и клиентских функций. Благодаря этому изменение структуры запроса или ответа быстрее обнаруживается на этапе сборки.

3.4 API gateway

API gateway является границей backend-части. Он предоставляет frontend HTTP-интерфейс в формате JSON и скрывает внутренние gRPC-сервисы. Для работы с фракталами предусмотрены маршруты получения списка фракталов, получения фрактала по идентификатору, запуска генерации, подписки на события задачи, получения списка функций и получения PNG-изображения. Для пользовательских сценариев предусмотрены маршруты регистрации, входа, обновления токена, получения и изменения профиля.

В gateway внешний API описан в файле `openapi.yaml`. По этой спецификации OpenAPI Generator формирует Java-модели и API-интерфейсы, которые затем реализуются контроллерами. Это снижает вероятность расхождения между документацией, серверной реализацией и клиентским кодом.

Внутри gateway слой HTTP-контроллеров отделён от слоя вызовов downstream-сервисов. Контроллер `FractalsController` принимает запросы на маршрутах `/api/v1/fractals`, `/api/v1/fractals/gen`,

/api/v1/fractals/functions, /api/v1/fractals/gen/{id}/events и /api/v1/fractals/images/{name}. Сервисный слой использует gRPC-stub'ы generator: блокирующий stub применяется для коротких операций, а асинхронный stub — для подписки на состояние задачи.

Пользовательские маршруты реализует UsersController. При регистрации gateway вызывает метод Register IAM-сервиса, при входе — метод Login, возвращает frontend access token в теле ответа и устанавливает refresh token в cookie. При обновлении профиля gateway вызывает UpdateUser, а при запросе изображения профиля — GetImage. Такой подход оставляет внешний контракт удобным для браузера, но не раскрывает frontend внутреннюю protobuf-схему IAM.

Для единообразной обработки ошибок gateway преобразует исключения gRPC в HTTP-ответы. Например, NOT_FOUND становится кодом 404, INVALID_ARGUMENT — кодом 400, UNAUTHENTICATED — кодом 401, а ALREADY_EXISTS — кодом 409. Frontend получает структурированный объект ошибки с кодом, статусом и сообщением и может показывать его в форме или уведомлении.

Gateway также отвечает за передачу пользовательского контекста. JWT приходит от frontend в HTTP-заголовке Authorization, после чего gateway передаёт токен во внутренние gRPC metadata. В результате generator может связать созданный фрактал с пользователем, не имея прямого HTTP-взаимодействия с браузером.

3.5 IAM

IAM выделен в отдельный микросервис, чтобы логика пользователей и токенов не смешивалась с генерацией изображений. Сервис предоставляет внутренний gRPC API для регистрации, входа, обновления токена, получения пользователя, изменения профиля и чтения изображения профиля. Внешние HTTP-маршруты остаются на gateway, поэтому frontend не зависит от конкретной реализации IAM.

Аутентификация построена на JWT [19]. После успешного входа IAM выдаёт access token и refresh token. Access token используется frontend при обращении к gateway, а refresh token позволяет обновлять пользовательскую сессию без повторного ввода пароля. Токен подписывается алгоритмом HMAC и содержит идентификатор пользователя в поле sub, имя пользователя и тип токена. Downstream-сервисы используют идентификатор как связку между действиями пользователя и данными системы.

Для IAM предусмотрена отдельная модель пользователей. В рамках текущих требований она состоит из одной таблицы users, где хранятся идентификатор пользователя, уникальное имя пользователя, bcrypt-хеш пароля и имя объекта изображения профиля. Идентификатор пользователя используется как subject JWT и передаётся downstream-сервисам для

связывания пользовательских действий с историей генераций. ER-диаграмма IAM-сервиса приведена на рисунке 3.7.

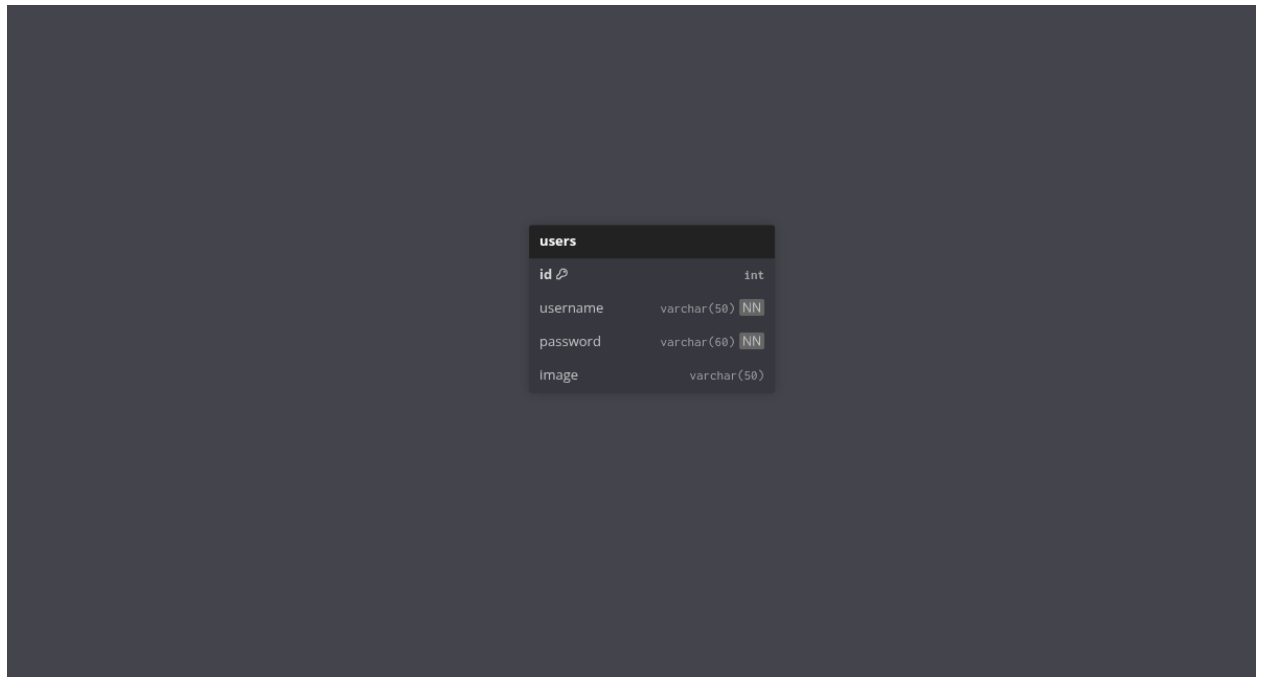


Рисунок 3.7 – ER-диаграмма IAM-сервиса

Сервисная логика IAM разделена на три направления. AuthService регистрирует пользователя, проверяет пароль и выпускает пару токенов. UserService получает профиль, обновляет имя пользователя, пароль и изображение. ImageService читает изображение из объектного хранилища. Пароль не сохраняется в открытом виде: перед записью в PostgreSQL он проходит проверку сложности и хешируется с помощью bcrypt. Изображение профиля передаётся как base64-строка, декодируется, проверяется по формату и размерам, после чего сохраняется в MinIO bucket images; в таблице users остаётся только имя объекта.

Для защищённых операций IAM извлекает идентификатор пользователя из access token. gRPC-interceptor читает metadata Authorization, проверяет токен и помещает user_id в контекст запроса. Метод обновления профиля использует именно этот идентификатор, поэтому пользователь не может передать произвольный id другого профиля во внешнем запросе.

3.6 Generator

Generator отвечает за вычислительно тяжёлую часть системы: создание фрактального пламени, хранение параметров генерации, хранение результата и выдачу истории работ. Внешние клиенты не обращаются к нему напрямую; доступ выполняется через API gateway по внутреннему

gRPC-контракту.

Основной контракт `generator` описан в файле `generator.proto`. Сервис `Fractals` предоставляет следующие операции:

- `GetAll` — получение списка фракталов с пагинацией, сортировкой и фильтрацией по `user_id`;
- `GetById` — получение одного фрактала по идентификатору;
- `Generation` — создание задачи генерации;
- `SubscribeToTask` — серверный поток состояния задачи;
- `GetFunctions` — получение доступных имён вариационных функций;
- `GetImage` — чтение изображения из S3-совместимого хранилища.

Запрос генерации содержит ширину и высоту изображения, число итераций, уровень симметрии, параметр гамма-коррекции, список вариационных функций и список аффинных преобразований. Перед постановкой задачи в очередь `generator` проверяет ограничения параметров: максимальную ширину, высоту, число итераций, положительный уровень симметрии, наличие хотя бы одной функции и одного набора аффинных параметров.

Сервис также содержит каталог поддерживаемых функций фрактального пламени. По коду `generator` доступны функции `linear`, `sinusoidal`, `spherical`, `swirl`, `horseshoe`, `polar`, `disc`, `heart` и `eyefish`. Каждая функция представлена отдельной моделью, что упрощает добавление новых вариаций без изменения внешнего API.

ER-диаграмма сервиса генерации показана на рисунке 3.8.

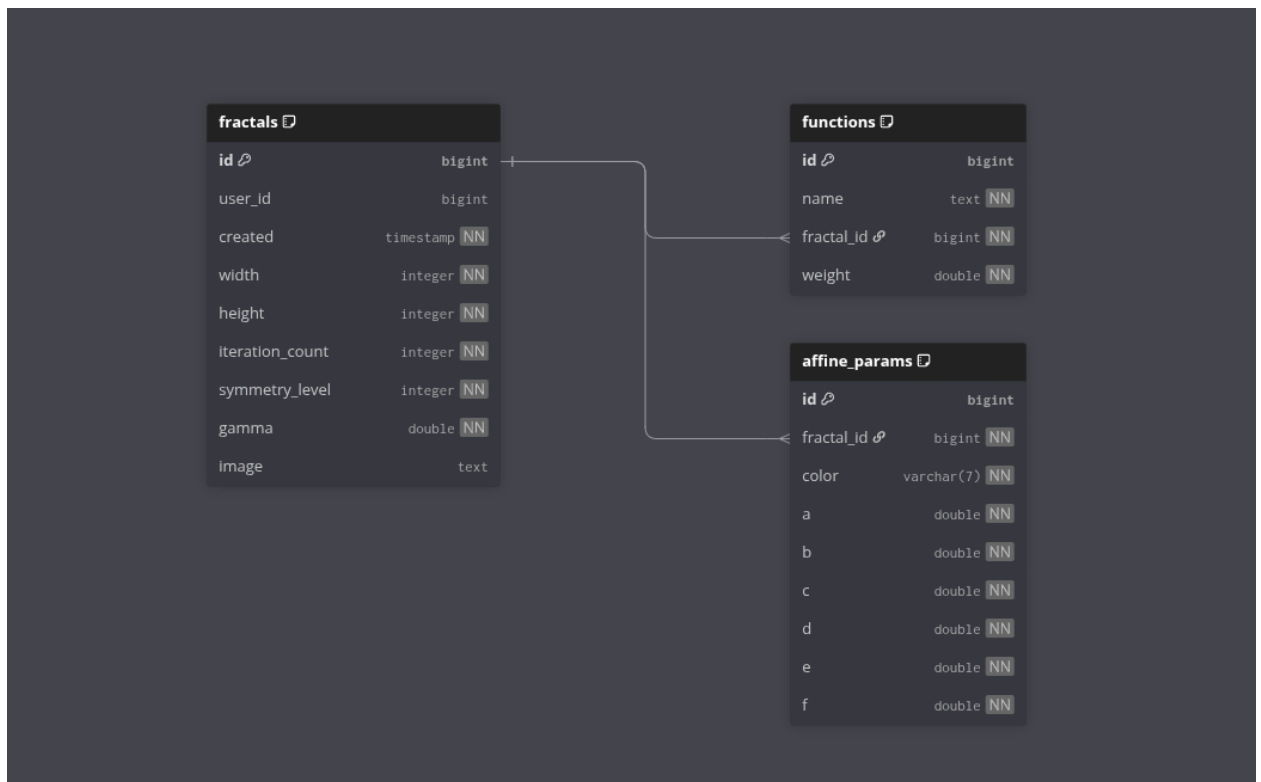


Рисунок 3.8 – ER-диаграмма сервиса генерации

Модель данных generator строится вокруг истории созданных фракталов. По миграциям Liquibase используются три таблицы: `fractals`, `functions` и `affine_params`. Таблица `fractals` хранит общие параметры результата: идентификатор, идентификатор пользователя, дату создания, размеры изображения, число итераций, уровень симметрии, гамму и имя файла изображения. Таблица `functions` хранит набор вариационных функций и веса для конкретного фрактала. Таблица `affine_params` хранит цвет и коэффициенты аффинного преобразования a , b , c , d , e , f .

Связи имеют тип «один ко многим»: один фрактал содержит несколько функций и несколько наборов аффинных параметров. Для дочерних таблиц настроено каскадное удаление по внешнему ключу `fractal_id`, поэтому удаление фрактала не оставляет несвязанных параметров. Для ускорения выборки созданы индексы по `fractals.user_id`, `functions.fractal_id` и `affine_params.fractal_id`.

Метаданные фракталов хранятся в PostgreSQL, а бинарные PNG-файлы вынесены в MinIO — локальное S3-совместимое объектное хранилище [18]. Такое разделение уменьшает нагрузку на реляционную базу данных и упрощает выдачу файлов. Generator использует AWS S3 SDK, создаёт bucket `images` при запуске, сохраняет PNG с типом `image/png` и возвращает имя файла, которое затем хранится в таблице `fractals`.

При запросе изображения gateway вызывает gRPC-метод `GetImage`; generator читает объект из MinIO и возвращает байтовое содержимое. Для

frontend это остаётся обычным HTTP-ответом с изображением, а внутренняя реализация хранения скрыта за gateway.

3.6.1 Генерация фрактального пламени строится как итерационный процесс построения изображения по набору аффинных преобразований и вариационных функций [24]. Пользователь передаёт размеры изображения, количество итераций, уровень симметрии, гамму, веса функций и коэффициенты аффинных преобразований. Эти параметры сохраняются как часть доменной модели фрактала, после чего на их основе создаётся объект задачи `GenerationTask`. Фрагменты кода генератора и сервиса генерации приведены в приложении А.

Внутри рендера выбирается начальная случайная точка. На каждой итерации выбирается одно аффинное преобразование и одна вариационная функция. Аффинное преобразование переносит точку в новое положение, а вариационная функция применяет нелинейное преобразование. Затем точка дополнительно обрабатывается с учётом уровня симметрии: для каждого симметричного поворота рассчитываются координаты пикселя на плоскости изображения.

Первые итерации используются как прогрев и не записываются в изображение, чтобы начальное случайное состояние меньше влияло на результат. После прогрева координаты точки преобразуются в координаты пикселя, цвет смешивается с цветом текущего аффинного преобразования, а счётчик посещений пикселя увеличивается. Чем чаще точка попадает в область изображения, тем сильнее эта область проявляется после нормализации.

Для ускорения вычислений `generator` делит работу между несколькими потоками. Класс `Generator` создаёт пул потоков, а каждый экземпляр `Render` выполняет свою часть итераций над общей моделью изображения. Доступ к пиксельной матрице синхронизируется, поскольку несколько потоков могут одновременно обновлять один и тот же результат. После завершения потоков выполняется гамма-коррекция: счётчики посещений переводятся в логарифмическую нормализацию, после чего цвет пикселей усиливается с учётом параметра `gamma`.

Результат генерации сохраняется в MinIO как PNG-файл, а в PostgreSQL остаётся ссылка на объект и параметры, достаточные для отображения истории. Такой подход делает результат воспроизводимым на уровне параметров и одновременно не перегружает реляционную базу бинарными данными.

3.6.2 Генерация фрактала может занимать заметное время, поэтому пользователь не должен ждать завершения HTTP-запроса без обратной связи. Для этого генерация реализована как асинхронная задача. Метод `Generation` в `generator` сохраняет параметры, создаёт `GenerationTask`, добавляет её в очередь и сразу возвращает начальное состояние задачи с идентификатором фрактала и текущим прогрессом.

Внутри `GeneratationService` используется очередь `BlockingQueue` и набор ожидающих задач `ConcurrentHashMap`. При запуске сервиса создаётся фиксированный пул `worker`-потоков. Каждый `worker` берёт задачу из очереди, вызывает `generator.process(task)`, после завершения сохраняет PNG-изображение в `MinIO`, обновляет запись фрактала в `PostgreSQL` и удаляет задачу из списка ожидающих.

Прогресс считается в `GenerationTask`. Внутри задачи хранится счётчик выполненных итераций `AtomicLong`; поток рендера периодически увеличивает его пакетами итераций. Метод `toTaskState()` формирует `protobuf`-сообщение `TaskState`: идентификатор фрактала, число прогресса от 0 до 1 и `base64`-предпросмотр текущего изображения.

Рассылка обновлений выполняется через `scheduler Spring`. Метод `tasksUpdate()` помечен аннотацией `@Scheduled(fixedDelayString = "${app.tasks-update-delay})`. `Scheduler` проходит по ожидающим задачам и вызывает `sendUpdate(task)`. Если для задачи есть подписчик, сервис отправляет очередное `TaskState` в `gRPC stream`.

`Gateway` подписывается на `gRPC`-метод `SubscribeToTask` асинхронным `stub`-ом. Каждое сообщение `TaskState`, пришедшее от `generator`, преобразуется в модель внешнего API и отправляется браузеру через `SseEmitter`. Для `frontend` это выглядит как поток `text/event-stream`, то есть `Server-Sent Events` [25]. Когда прогресс достигает 1, `gateway` завершает `SSE`-поток. Если `gRPC stream` завершается ошибкой, `SseEmitter` также завершается с ошибкой, чтобы клиент мог показать пользователю корректное состояние.

3.7 Развёртывание

Развёртывание выполняется через `Docker Compose` [20]. Каждый сервис запускается в отдельном контейнере, а инфраструктурные зависимости поднимаются рядом с приложением. В состав `Compose`-проекта входят контейнеры `frontend`, `gateway`, `generator`, `iam`, две базы `PostgreSQL` для `generator` и `IAM`, два `MinIO`-хранилища для изображений и контейнер миграций `IAM`.

Для `generator` используются `PostgreSQL` и `MinIO`; контейнер `generator` получает адрес `PostgreSQL` через переменную `POSTGRES_URL`, адрес `MinIO` через `MINIO_URL`, а секрет `JWT` через `JWT_SIGNING_KEY`. `Gateway` получает адрес `generator` через `GENERATOR_URL` и адрес `IAM` через `IAM_URL`. `IAM` использует отдельные переменные подключения к `PostgreSQL`, `MinIO` и параметры подписи `access/refresh token`. `Frontend` собирается в отдельном `Dockerfile`: сначала по `OpenAPI`-спецификации генерируется `TypeScript` `Axios`-клиент, затем `Vite` собирает статические файлы, после чего они отдаются контейнером `Nginx`.

`Compose`-сценарий содержит `healthcheck`-и для базы данных, `MinIO`,

gateway и generator. Это снижает вероятность запуска приложения до готовности зависимостей и делает локальное окружение воспроизводимым. На сервере запускается тот же Compose-проект, что используется при локальной проверке; отличаются только значения переменных окружения, доменное имя и настройки обратного прокси.

Развёртывание на VPS выполняется по SSH с использованием Docker Compose context. На рабочей машине создаётся удалённый Docker context, указывающий на Docker Engine сервера через SSH, после чего команды Compose выполняются для удалённого окружения без ручного копирования контейнеров и запуска сервисов через интерактивную сессию. При таком подходе локально остаются исходный код, compose-файл и файл окружения, а сборка образов и запуск контейнеров выполняются на сервере. Это уменьшает различия между локальным и серверным запуском и позволяет использовать один и тот же сценарий обновления.

Финальная версия системы разворачивается в Docker-контейнерах на VPS. В составе развёртывания применяется обратный прокси для маршрутизации запросов и разграничения доступа к сервисам: публичными остаются frontend и HTTP API gateway.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1 Стратегия тестирования

Тестирование системы проводилось на нескольких уровнях: модульном, интеграционном и системном. На модульном уровне проверялась корректность отдельных компонентов генерации и обработки параметров. Интеграционные тесты подтверждали корректность взаимодействия микросервисов, а системные тесты моделировали реальную работу пользователя. Подход соответствует классическим рекомендациям по организации тестирования программного обеспечения [26].

Для проверки критических сценариев использовались контрольные наборы параметров рендера и заранее подготовленные эталонные результаты. Это позволило выявлять регрессии при изменениях алгоритмов и конфигурации.

4.2 Сценарии тестирования

Ниже приведены основные сценарии, подтверждающие работоспособность системы:

- 1 Регистрация и вход пользователя: создание записи в IAM, получение access token и refresh token.
- 2 Проверка JWT: передача access token из frontend в gateway и дальнейшая передача токена во внутренний gRPC-вызов.
- 3 Создание задачи генерации: отправка параметров фрактала через HTTP API gateway и получение идентификатора задачи.
- 4 Подписка на прогресс: получение событий text/event-stream, сформированных gateway на основе gRPC stream generator.
- 5 Сохранение результата: запись параметров фрактала в PostgreSQL и PNG-изображения в MinIO.
- 6 Просмотр истории: получение списка фракталов с пагинацией, сортировкой и фильтрацией по пользователю.
- 7 Получение изображения: запрос изображения через gateway и чтение объекта из MinIO сервисом generator.
- 8 Проверка ограничений параметров: отказ generator при превышении максимальных размеров или количества итераций.

Результаты тестирования подтвердили корректность работы ключевых функций и соответствие заявленным требованиям.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта получена проектная и программная основа веб-системы для генерации изображений фрактального пламени. Система закрывает основной недостаток рассмотренных аналогов: вместо локального desktop-инструмента пользователь получает веб-интерфейс с запуском генерации, просмотром прогресса, сохранением результата, общей галереей последних работ и личной историей.

Главным инженерным результатом стала микросервисная архитектура из четырёх компонентов: frontend, API gateway, IAM и generator. Frontend работает с gateway по HTTP, а gateway взаимодействует с IAM и generator по gRPC. Такое разделение позволило отделить пользовательский интерфейс от внутренней логики аутентификации и вычислений. Внешний API зафиксирован OpenAPI-спецификацией, а внутренний контракт generator описан protobuf-схемой, что уменьшает риск расхождения между клиентом, gateway и backend-сервисами.

Отдельным результатом стала реализация сервиса generator. Он принимает параметры фрактального пламени, проверяет ограничения, сохраняет модель фрактала, ставит задачу в очередь, выполняет многопоточный рендер, применяет гамма-коррекцию и сохраняет PNG-изображение в MinIO. Метаданные результата сохраняются в PostgreSQL, а связь с пользователем выполняется через идентификатор из JWT. Код генератора и сервиса генерации приведён в приложении А.

Для пользовательского опыта был решён сценарий длительной операции генерации. Generator хранит состояние задачи, scheduler периодически формирует обновления прогресса, gRPC stream передаёт их в gateway, а gateway преобразует поток во внешний SSE-канал для браузера. Благодаря этому пользователь видит не только факт запуска задачи, но и промежуточный предпросмотр изображения вместе с числовым прогрессом.

Таким образом, цель курсового проекта достигнута: спроектирована и реализована основа распределённого приложения, поддерживающего регистрацию и вход пользователя, генерацию фракталов, передачу прогресса, сохранение истории, выдачу изображений и просмотр результатов через веб-интерфейс. Полученное решение остаётся расширяемым: в дальнейшем можно добавить новые вариационные функции, роли пользователей, очередь фоновых задач, более гибкий редактор параметров и дополнительные способы публикации готовых изображений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Apophysis: Fractal Flame Editor [Электронный ресурс]. — Режим доступа : <https://sourceforge.net/projects/apophysis/>.
- [2] JWildfire: Fractal Flame Generator (Repository) [Электронный ресурс]. — Режим доступа : <https://github.com/thargor6/JWildfire>.
- [3] Chaotica Fractal Flame Software [Электронный ресурс]. — Режим доступа : <https://www.chaoticafractals.com/>.
- [4] Booch, G. Object-Oriented Analysis and Design with Applications / G. Booch. — 3 edition. — Addison-Wesley Professional, 2007.
- [5] Martin, Robert C. Agile Software Development: Principles, Patterns, and Practices / Robert C. Martin. — Prentice Hall, 2002.
- [6] Design Patterns: Elements of Reusable Object-Oriented Software / E. Gamma, R. Helm, R. Johnson, J. Vlissides. — Addison-Wesley, 1994.
- [7] Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. — Addison-Wesley, 2003.
- [8] gRPC Documentation [Электронный ресурс]. — Режим доступа : <https://grpc.io/docs/>.
- [9] Schildt, H. Java: The Complete Reference, Ninth Edition / H. Schildt. — McGraw-Hill Education, 2014. — Pp. 17 – 23.
- [10] Walls, Craig. Spring in Action / Craig Walls. — 6 edition. — Manning Publications, 2022.
- [11] Gradle User Manual [Электронный ресурс]. — Режим доступа : <https://docs.gradle.org/current/userguide/userguide.html>.
- [12] The Go Programming Language Documentation [Электронный ресурс]. — Режим доступа : <https://go.dev/doc/>.
- [13] TypeScript Documentation [Электронный ресурс]. — Режим доступа : <https://www.typescriptlang.org/docs/>.
- [14] Vue.js Documentation [Электронный ресурс]. — Режим доступа : <https://vuejs.org/guide/introduction.html>.
- [15] Tailwind CSS Documentation [Электронный ресурс]. — Режим доступа : <https://tailwindcss.com/docs>.
- [16] Axios Documentation [Электронный ресурс]. — Режим доступа : <https://axios-http.com/docs/intro>.
- [17] PostgreSQL Documentation [Электронный ресурс]. — Режим доступа : <https://www.postgresql.org/docs/>.
- [18] MinIO Object Store Documentation [Электронный ресурс]. — Режим доступа : <https://min.io/docs/minio/linux/index.html>.
- [19] Jones, M. JSON Web Token (JWT): Tech. Rep. RFC 7519 / M. Jones, J. Bradley, N. Sakimura: IETF, 2015. — Режим доступа : <https://www.rfc-editor.org/rfc/rfc7519>.
- [20] Docker Compose Documentation [Электронный ресурс]. — Режим доступа : <https://docs.docker.com/compose/>.

- [21] Visual Studio Code Documentation [Электронный ресурс]. — Режим доступа : <https://code.visualstudio.com/docs>.
- [22] Круг, Стив. Не заставляйте меня думать / Стив Круг. — ”Эксмо”, 2019.
- [23] OpenAPI Specification [Электронный ресурс]. — Режим доступа : <https://spec.openapis.org/oas/latest.html>.
- [24] Draves, S. The Fractal Flame Algorithm / S. Draves, E. Reckase // The Flame Algorithm. — 2003. — 41 P.
- [25] Server-sent events — MDN Web Docs [Электронный ресурс]. — Режим доступа : https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events.
- [26] Myers, Glenford J. The Art of Software Testing / Glenford J. Myers, Corey Sandler, Tom Badgett. — 3 edition. — Wiley, 2011.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

GeneratationService.java

```
package com.fractalflame.generator.service;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

import com.fractalflame.generator.exception.GeneratorException;
import com.fractalflame.generator.exception.TaskNotFoundException;
import com.fractalflame.generator.properties.GeneratorProperties;
import com.fractalflame.generator.proto.TaskState;
import com.fractalflame.generator.repository.FractalsRepository;
import com.fractalflame.generator.utils.GenerationTask;
import com.fractalflame.generator.utils.Generator;

import io.grpc.Context;
import io.grpc.stub.StreamObserver;
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

@Slf4j
@Service
@RequiredArgsConstructor
public class GeneratationService {
    private final GeneratorProperties properties;
    private final Generator generator;
    private final StorageService storageService;
    private final FractalsRepository fractalsRepository;

    private final BlockingQueue<GenerationTask> taskQueue = new
        LinkedBlockingQueue<>();
    private final Map<Long, GenerationTask> pendingTasks = new
        ConcurrentHashMap<>();

    private ExecutorService workers;

    @PostConstruct
```

```

void init() {
    workers = Executors.newFixedThreadPool(properties.getCount());

    for (int i = 0; i < properties.getCount(); i++) {
        workers.submit(() -> {
            while (!Thread.currentThread().isInterrupted()) {
                try {
                    var task = taskQueue.take();
                    generator.process(task);
                    sendUpdate(task);
                    pendingTasks.remove(task.getFractal().getId());

                    var fractal = task.getFractal();
                    fractal.setImage(storageService.saveImage(UUID.
                        randomUUID().toString(),
                        task.getImage()));

                    synchronized (fractalsRepository) {
                        fractalsRepository.save(fractal);
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    throw new GeneratorException(e);
                }
            }
        });
    }
}

@Scheduled(fixedDelayString = "${app.tasks-update-delay}")
void tasksUpdate() {
    for (var task : pendingTasks.values()) {
        sendUpdate(task);
    }
}

@PreDestroy
void free() {
    workers.shutdown();
    try {
        if (!workers.awaitTermination(5, TimeUnit.SECONDS)) {
            workers.shutdownNow();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new GeneratorException(e);
    }
}

public void addTask(GenerationTask task) {
    taskQueue.add(task);
    pendingTasks.put(task.getFractal().getId(), task);
}

```

```

    }

    public void subscribeToTask(Long id, StreamObserver<TaskState>
        responseObserver) {
        if (pendingTasks.containsKey(id)) {
            var task = pendingTasks.get(id);
            task.setResponseObserver(responseObserver);
            Context.current().addListener(
                context -> {
                    log.info("Subscription cancelled for task {}", id);
                    task.setResponseObserver(null);
                },
                Runnable::run);
        } else {
            throw new TaskNotFoundException(String.format("Task with id=%s
                not pending!", id));
        }
    }

    private void sendUpdate(GenerationTask task) {
        var responseObserver = task.getResponseObserver();
        if (responseObserver != null) {
            log.atInfo().addKeyValue("fractal_id", task.getFractal().getId())
                .log("send task update");
            try {
                responseObserver.onNext(task.toTaskState());
            } catch (Exception e) {
                log.warn("Subscriber disconnected", e);
                task.setResponseObserver(null);
            }
        }
    }
}

```

GenerationTask.java

```

package com.fractalflame.generator.utils;

import java.util.concurrent.atomic.AtomicLong;

import com.fractalflame.generator.entity.Fractal;
import com.fractalflame.generator.model.Image;
import com.fractalflame.generator.proto.TaskState;

import io.grpc.stub.StreamObserver;
import lombok.AccessLevel;
import lombok.Getter;
import lombok.Setter;
import lombok.extern.slf4j.Slf4j;

@Slf4j
@Getter

```

```

public class GenerationTask {
    private Fractal fractal;
    private Image image;

    @Setter
    private StreamObserver<TaskState> responseObserver;

    @Getter(AccessLevel.NONE)
    private final AtomicLong iterations = new AtomicLong();

    public GenerationTask(Fractal fractal) {
        this.fractal = fractal;
        image = new Image(fractal.getWidth(), fractal.getHeight());
    }

    public void addIterations(Long iterations) {
        this.iterations.addAndGet(iterations);
    }

    public synchronized TaskState toTaskState() {
        var progress = (double) iterations.get() / (fractal.
            getIterationCount() - 10);
        if (progress > 1) {
            progress = 1.0;
        }
        synchronized (image) {
            return TaskState.newBuilder()
                .setFractalId(fractal.getId())
                .setProgress(progress)
                .setPreview(ImageWriter.toBase64(image))
                .build();
        }
    }
}

```

Generator.java

```

package com.fractalflame.generator.utils;

import com.fractalflame.generator.mapper.AffineMapper;
import com.fractalflame.generator.mapper.FunctionMapper;
import com.fractalflame.generator.model.Image;
import com.fractalflame.generator.properties.GeneratorProperties;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

import org.springframework.stereotype.Component;

```

```

@Component
@Slf4j
@RequiredArgsConstructor
public class Generator {
    private final GeneratorProperties properties;
    private final FunctionMapper functionMapper;
    private final AffineMapper affineMapper;

    public void process(GenerationTask task) {
        var fractal = task.getFractal();

        log.atInfo().addKeyValue("fractal_id", fractal.getId()).log("Start
            render...");

        try (var executor = Executors.newFixedThreadPool(properties.
            getThreads())) {
            for (int i = 0; i < properties.getThreads(); i++) {
                executor.execute(new Render(task,
                    fractal.getIterationCount() / properties.getThreads(),
                    fractal.getSymmetryLevel(),
                    functionMapper.toFunctionModelList(fractal.
                        getFunctions()),
                    affineMapper.toAffineTransformationList(fractal.
                        getAffineParams())));
            }
            executor.shutdown();
            try {
                if (!executor.awaitTermination(30, TimeUnit.MINUTES)) {
                    executor.shutdownNow();
                }
            } catch (InterruptedException e) {
                executor.shutdownNow();
                Thread.currentThread().interrupt();
            }
            log.atInfo().addKeyValue("fractal_id", fractal.getId()).log("
                Render complete!");
        }

        correction(task.getImage(), fractal.getGamma());
    }

    private void correction(Image image, Double gamma) {
        log.info("Start correction...");

        var pixels = image.getPixels();

        double max = 0;

        for (var row : pixels) {
            for (var pixel : row) {
                if (pixel != null) {
                    pixel.setNormal(Math.log10(pixel.getCounter().sum()));
                }
            }
        }
    }
}

```

```

        max = Math.max(pixel.getNormal(), max);
    }
}

if (max <= 0) {
    return;
}

for (var row : pixels) {
    for (var pixel : row) {
        if (pixel != null) {
            var normal = pixel.getNormal();
            normal /= max;
            pixel.setNormal(normal);
            pixel.multiply(Math.pow(normal, 1.0 / gamma));
        }
    }
}
}
}

```

Render.java

```

package com.fractalflame.generator.utils;

import com.fractalflame.generator.model.AffineTransformation;
import com.fractalflame.generator.model.Pixel;
import com.fractalflame.generator.model.Point;
import com.fractalflame.generator.model.functions.FunctionModel;

import lombok.AllArgsConstructor;
import lombok.extern.slf4j.Slf4j;

import java.util.Comparator;
import java.util.List;
import java.util.Random;

@Slf4j
@AllArgsConstructor
public class Render implements Runnable {
    private final Random random = new Random(System.currentTimeMillis());
    private final GenerationTask task;
    private final int iterationsCount;
    private final int symmetryLevel;
    private final List<FunctionModel> functions;
    private final List<AffineTransformation> affineTransformations;

    @Override
    public void run() {
        synchronized (log) {
            log.info("[Thread: {}] Start generation...", this.hashCode());

```

```

}

var image = task.getImage();

var width = image.getWidth();
var height = image.getHeight();
var pixels = image.getPixels();

double xMax = width > height ? (double) width / height : 1;
double yMax = width < height ? (double) height / width : 1;
double xMin = -xMax;
double yMin = -yMax;
double theta = 2 * Math.PI / symmetryLevel;

var point = new Point(random.nextDouble(xMin, xMax), random.
    nextDouble(yMin, yMax));
var color = Pixel.genColor(random);

long iterationsDelta = -20;

for (int i = -20; i < iterationsCount; i++, iterationsDelta++) {
    var currentAffine = getRandomAffine();
    point = getRandomFunction().transform(currentAffine.transform(
        point));

    for (int j = 0; j < symmetryLevel; j++) {
        double angle = j * theta;

        double xRot = point.x() * Math.cos(angle) - point.y() * Math.
            sin(angle);
        double yRot = point.x() * Math.sin(angle) + point.y() * Math.
            cos(angle);

        point = new Point(xRot, yRot);

        int x = width - (int) ((xMax - point.x()) / (xMax - xMin) *
            width);
        int y = height - (int) ((yMax - point.y()) / (yMax - yMin) *
            height);

        color.add(currentAffine.getColor()).multiply(0.5);

        if (i >= 0 && 0 < x && x < width && 0 < y && y < height) {
            var pixel = new Pixel(color);
            synchronized (image) {
                if (pixels[y][x] != null) {
                    pixel.setCounter(pixels[y][x].getCounter());
                }
                pixels[y][x] = pixel;
            }
            pixels[y][x].getCounter().increment();
        }
    }
}

```

```

    }

    if (iterationsDelta >= 1000) {
        task.addIterations(iterationsDelta);
        iterationsDelta = 0;
    }
}

task.addIterations(iterationsDelta);

synchronized (log) {
    log.info("[Thread: {}] Generation complete!", this.hashCode());
}

}

private AffineTransformation getRandomAffine() {
    return affineTransformations.get(random.nextInt(
        affineTransformations.size()));
}

private FunctionModel getRandomFunction() {
    double sum = functions.stream().mapToDouble(FunctionModel::getWeight)
        .sum();
    double n = 0;
    double rNum = random.nextDouble(sum);
    for (var function : functions) {
        n += function.getWeight();
        if (n >= rNum) {
            return function;
        }
    }
    return functions.stream().max(Comparator.comparing(FunctionModel::
        getWeight)).orElseGet(functions::getFirst);
}
}

```